

AWS Developer Exam: Domain 2 – Authentication & Authorization

Overview

This domain covers implementing authentication and authorization for applications and AWS services. Key themes include:

- Defining trust boundaries and system security configurations

- Authentication and authorization mechanisms

- Policy enforcement points (e.g., Web Application Firewalls, API Gateways)

- Managing identities across Regions, Edge Locations, Availability Zones, and Local Zones

- Understanding the resiliency of AWS services (globally resilient, regionally resilient, or AZ-resilient)

- Understanding how AWS could fail and how to add infrastructure protection is just as important as knowing how services work in a healthy state.

AWS Accounts & IAM Fundamentals

AWS accounts are the foundation of access control in AWS. IAM (Identity and Access Management) is a **global AWS service** — meaning it is not scoped to a region or availability zone.

IAM Best Practices

- Protect the root user** — enable MFA and avoid using it for day-to-day tasks.

- Apply the **principle of least privilege** — grant only the permissions needed to perform a task.

- Use **MFA (Multi-Factor Authentication)** for all privileged users.

Principal Entities

A principal is an entity (user, role, service, or application) that can make requests to AWS. Principals interact with AWS resources through identity-based and resource-based policies. You need to know how to implement permissions across:

- Policies and accounts

- Policies and users

- Policies and groups

- Identity-based vs. resource-based policies

IAM Policy Types

Policy types are broadly categorized as **permissions policies** or **permissions boundaries**.

Policy Type	Description	Grants Permissions?
AWS Managed Policies	Pre-built policies created and managed by AWS	Yes
Customer Managed Policies	Policies you create and manage in your account	Yes
Inline Policies	Embedded directly into a single IAM user, group, or role	Yes
Permission Boundaries	Sets the <i>maximum</i> permissions an identity-based policy can grant to a user or role	No (limits only)
Service Control Policies (SCPs)	Applied at the AWS Organization level; limits what identity-based and resource-based policies can grant	No (limits only)
Session Policies	Passed during an AssumeRole or federation call; limits permissions for that specific session	No (limits only)

Key Exam Distinction: SCPs and session policies both *limit* permissions — they do **not** grant them. An SCP alone will never give a user access to anything.

Diving Deeper: Resource-Based vs. Identity-Based Policies

Identity-based policies are attached to IAM users, groups, or roles.

Resource-based policies are attached directly to a resource (e.g., an S3 bucket policy) and specify who can access that resource.

IAM Roles

IAM roles are a critical concept and will appear frequently on the exam. A role is an identity with permissions that can be assumed by users, services, or applications.

When to Use Roles vs. Other Options

IAM roles — use when applications or services (like ECS tasks) need permissions to access AWS resources.

IAM groups — used to manage permissions for multiple IAM users together; cannot be assumed by services.

Service-linked roles — predefined roles linked directly to an AWS service (e.g., ECS itself), not to tasks running within that service.

Exam Example: If your application is hosted in ECS and needs permissions for 4 separate tasks, use **IAM roles** — one per task. Service-linked roles are tied to the ECS service itself, not individual task executions.

Roles can also delegate access to:

- Users in a different AWS account

- AWS services acting on your behalf

- External applications that don't have direct AWS credentials

Access Control Models

RBAC — Role-Based Access Control

The traditional IAM model where permissions are defined based on a user's job function. Implemented by creating and attaching policies to IAM users, groups, or roles.

ABAC — Attribute-Based Access Control

A more scalable model where permissions are defined based on **tags or attributes** attached to identities and resources. Instead of writing many policies for different roles, you write policies that grant access based on matching tags (e.g., `Department=Finance`). ABAC scales well across large organizations and multi-account environments.

AWS Organizations & Service Control Policies (SCPs)

In AWS Organizations, you manage multiple AWS accounts centrally. SCPs are applied at the organization, OU (organizational unit), or account level and act as guardrails.

SCPs **limit** what identity-based and resource-based policies can grant to users or roles within accounts.

SCPs do **not** grant any permissions themselves.

To test and troubleshoot the impact of SCPs on IAM policies and resource policies, use the **IAM Policy Simulator**.

Exam Question: Which service helps you test the impact of SCPs on IAM and resource-based policies in AWS Organizations? → **IAM Policy Simulator** (not Systems Manager, Inspector, or Config).

IAM Identity Center

IAM Identity Center (formerly AWS SSO) is the recommended way to **scale secure access across multiple AWS accounts and applications**. It enables centralized management of SSO access and integrates with external identity providers.

AWS Directory Services

Some AWS services (like Amazon WorkSpaces) require a directory. AWS offers three directory options:

Service	Description	Best For
Simple AD	Basic Active Directory-compatible functions; supports 500–5,000 users	Simple use cases with minimal AD requirements
Managed Microsoft AD	Full AWS-managed Microsoft Active Directory; supports trust relationships with on-premises directories	Organizations needing real Microsoft AD features and on-prem integration
AD Connector	Proxy service that redirects directory requests to your on-premises AD; stores no directory data in the cloud	Organizations that want to use existing on-prem AD with AWS services without replicating data

Identity Federation

Identity federation allows users outside of AWS to authenticate using an external identity provider (IDP) and receive temporary AWS credentials. There are three types:

1. Cross-Account Role

A remote IDP or account is allowed to assume a role in your account and access your resources. Uses

`AssumeRole` via AWS STS.

2. SAML 2.0 Federation

Used primarily with on-premises directories like Microsoft Active Directory. Users authenticate with their corporate credentials and receive a SAML assertion, which is exchanged for temporary AWS credentials via STS.

API call: `AssumeRoleWithSAML`

3. Web Identity Federation

Uses public identity providers (Amazon, Google, Facebook). Users authenticate with their IDP and receive an identity token, which is exchanged for temporary AWS credentials via STS or Cognito.

API call: `AssumeRoleWithWebIdentity`

The Federation Flow

User authenticates with an external IDP.

IDP returns an authentication token or assertion.

Token is exchanged for **temporary AWS credentials** via **AWS STS**.

Temporary credentials map to an IAM role with defined permissions.

User accesses AWS resources using those temporary credentials.

AWS STS — Secure Token Service

STS provides **short-term, temporary credentials** to access AWS resources. Some services require a bearer token from STS before you can access resources programmatically.

Key STS API Calls

API Call	When to Use
<code>AssumeRole</code>	Assume a role in the same or different account; used for cross-account access
<code>AssumeRoleWithSAML</code>	Federation via SAML 2.0 (on-prem AD / corporate identity)
<code>AssumeRoleWithWebIdentity</code>	Federation via web IDPs (Amazon, Google, Facebook)
<code>GetFederationToken</code>	Used by a proxy application to get temporary credentials for distributed applications on a corporate network
<code>GetSessionToken</code>	Use when you want MFA to protect specific programmatic API operations

IAM Identity Providers (IDPs)

Instead of creating IAM users for external identities (partners, auditors, mobile app users), use **IAM Identity Providers**. This avoids distributing or embedding long-term AWS credentials.

IAM supports these federation standards:

SAML 2.0 — for enterprise/on-premises systems

OpenID Connect (OIDC) — for modern web and mobile applications

Amazon Cognito — AWS-native identity broker for web and mobile apps

Identity federation is especially useful for:

Mobile applications accessing AWS resources

Third-party audits requiring temporary, scoped access

Any scenario where you don't want to manage long-term IAM credentials

Exam Best Practice: For a mobile app using S3 and DynamoDB that needs signed AWS requests, the best approach is to request **temporary credentials via STS mapped to a least-privilege IAM role** — never embed long-term access keys in the app.

Amazon Cognito

Cognito is AWS's managed identity service for web and mobile applications. It acts as an **identity broker** between your app and web identity providers, requiring no additional custom code.

Two Main Components

User Pools

A **user directory** that handles sign-up and sign-in for your application.

Authenticates users and returns **tokens** (identity token, access token, refresh token).

Supports built-in sign-in as well as federation with social IDPs and SAML providers.

Use when you need to manage and authenticate your application's users.

Identity Pools (Federated Identities)

Grant users **temporary AWS credentials** to directly access AWS services (e.g., S3, DynamoDB).

Can federate identities from Cognito User Pools, social IDPs, SAML, or OIDC providers.

Use when your authenticated users need to interact with AWS resources directly.

Cognito Authentication Flow

User signs in via a User Pool (or external IDP).

User receives tokens from Cognito.

Tokens are exchanged with an Identity Pool for temporary AWS credentials via STS.

User accesses AWS services with scoped, temporary credentials.

Exam Note: Cognito User Pools = authentication (who you are). Identity Pools = authorization (what AWS resources you can access).

API Gateway & Lambda Authorizers

A **Lambda authorizer** is an API Gateway feature that uses a Lambda function to control access to your API.

It's useful when implementing custom authentication strategies.

Two Types of Lambda Authorizers

Token-Based (TOKEN Authorizer)

Receives the caller's identity via a **bearer token** (e.g., JWT, OAuth token).

Best for implementing OAuth or SAML-style authentication strategies.

The Lambda function validates the token and returns an IAM policy.

Request Parameter-Based (REQUEST Authorizer)

Receives caller identity through a combination of **headers, query string parameters, stage variables, and `$context` variables**.

More flexible; useful when identity info isn't in a single token.

Exam Question: Your API Gateway uses a Lambda authorizer and you need to implement a strategy similar to OAuth or SAML. Which authorizer type should you use? → **Token-based Lambda authorizer**, because it accepts bearer tokens like JWT or OAuth tokens.

Cross-Account Lambda Authorizer

You can use a Lambda function from a **different AWS account** as your authorizer by configuring cross-account permissions — useful in multi-account architectures.

Key Exam Tips & Practice Questions

IAM is global — not regional or zonal.

SCPs and session policies limit permissions; they never grant them.

Use IAM roles for ECS tasks, not service-linked roles (which belong to the ECS service itself).

IAM Policy Simulator is the tool to test SCP impact on IAM and resource-based policies in AWS Organizations.

Never embed long-term credentials in mobile or distributed applications — always use temporary credentials via STS.

Cognito User Pools = authentication; Identity Pools = AWS resource authorization.

Token-based Lambda authorizers are the right choice for OAuth/SAML bearer token strategies in API Gateway.

AD Connector is right when you want to proxy requests to on-prem AD without storing any directory data in AWS.

Know the difference between `GetFederationToken` (proxy apps on corporate networks) and `GetSessionToken` (MFA-protected API calls).

Be able to **read and interpret IAM policies** — you don't need to write code, but policy reading is tested.